Algoritmo

Ariana Galli

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a mathlab y permite el trabajo con matrices.

```
a=38
a=40
b=80
c=b-a
c
```

```
## [1] 40
```

```
y=40
alpha<-30
```

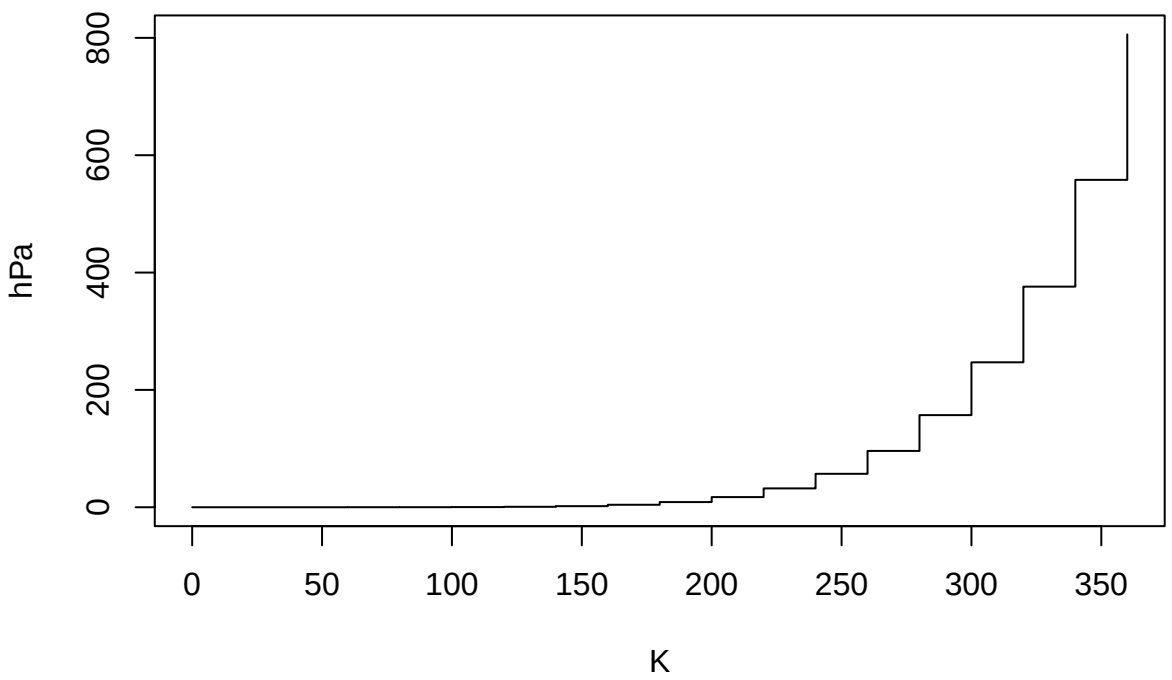
##uso de datasets o base de datos internos de R usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ depues del comando de informacion especifico y elegimos la columa de datos deseada

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:



##comando plot El comando plot es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar un comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

##Funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 18.324925 27.504491 18.585773  9.984460 26.453945 27.480440 14.316380
##   [8] 15.066565 14.174017 28.907189 23.613660 27.508794 16.857208 19.960063
##  [15] 23.930781 17.297530 26.982182  6.784891 19.619437 20.535353 14.786621
##  [22] 22.033781 17.177272 19.132240 24.825019 20.270959 14.536031 28.959745
##  [29] 16.782157 27.980255 18.894093 16.165847 21.538610 26.659414 14.481496
##  [36] 25.329952 21.687341 22.798961 22.741998 12.183289 26.630513 27.349391
##  [43] 26.447963 28.790064 17.021803 23.055870 22.097475 25.386193 23.545948
##  [50] 13.449720 15.474202 28.136545 17.877798  9.006336 26.690215 24.272058
##  [57] 30.315593 31.484933 14.175732 26.150073 23.808743 16.771998 31.207448
##  [64] 24.894441 30.525310 20.067125 19.658627 18.046143 25.777301 21.980035
##  [71] 17.403232 23.841611 25.046639 23.776092 16.419635 22.204954 19.175986
##  [78] 33.529740 23.512359 13.296645 23.219558 23.727749 19.805472 24.188682
##  [85] 26.273808 13.937121 19.987769 18.205544 24.680178 11.668040 12.060469
##  [92] 12.842513 16.875998 24.896902 27.165937 20.056898 28.833926 22.779176
##  [99] 32.780188 19.054584 27.443066 15.315330 17.153915 23.056134 27.654042
## [106] 19.630018 15.439484 32.021874 28.568440 11.764463 25.249355 27.693203
## [113] 32.828912 25.215472 25.978984 13.836762 25.953986 20.993979 17.391216
## [120] 31.798830 24.489434 15.941636 21.628433 24.019903 27.984985 18.816856
## [127] 17.522924 33.681884 21.636079 24.207004 16.465998 17.905571 14.570654
## [134] 17.014732 26.701699 14.420181 22.712188 20.129226 26.229787 15.646307
## [141] 18.944689 23.284277 25.506035 25.929845 23.333073 28.572449 28.344952
```

```
## [148] 26.175980 21.860045 29.446437 16.572644 14.185247 28.982003 17.185065
## [155] 20.453105 14.978191 20.463684 29.668571 20.634009 24.500843 24.114749
## [162] 18.161908 25.060381 21.584735 24.855115 29.688413 26.090118 22.212522
## [169] 16.577023 21.675063 17.558425 15.142197 30.201704 31.569938 19.739387
## [176] 18.197647 15.961790 14.724274 23.281729 21.916945 22.848125 30.206252
## [183] 18.614151 28.793918 26.642625 27.612908 14.926309 24.161422 27.209015
## [190] 24.843169 17.718682 17.324226 24.666047 20.397468 30.363062 19.223703
## [197] 26.373213 17.749349 23.233524 23.757542 19.193887 24.479160 22.084675
## [204] 21.044003 22.143079 20.668324 23.452802 26.079129 20.632133 22.716921
## [211] 16.098052 29.440354 13.091938 19.396818 31.925174 31.099196 23.021249
## [218] 29.623140 19.791476 13.600537 26.259743 25.252011 21.280501 16.548380
## [225] 26.048426 21.884726 16.857841 16.037353 24.372428 37.040780 13.674156
## [232] 13.341717 13.394950 22.830551 22.718671 18.588007 23.608686 18.984342
## [239] 21.533147 18.095159  8.322122 22.441469 20.017465 23.202315 20.593435
## [246] 30.246692 14.645014 19.919043 29.359623 14.504836 18.324229 28.318130
## [253] 12.576421 14.822119 20.402418 13.871087 23.476940 21.860390 23.092604
## [260] 19.299323 11.174393 23.314089 18.060945 29.502613 19.154170 22.753231
## [267] 20.114398 13.968689 19.073702 24.054706 34.678644 16.608063 21.390771
## [274] 17.996404 23.427309 18.632817 24.292538 22.719702 14.994297 21.985653
## [281] 22.174524 20.930270 15.877236 19.030328 24.176645 27.712474 14.782535
## [288] 23.488849 22.931966 21.924899 23.686783 13.334636 19.210524 24.549167
## [295] 27.834178 29.298658 20.744308 24.730464 12.532316 26.223355 27.552487
## [302] 23.398414 29.350115 17.831059 18.995562 22.283361 17.351921 19.576995
## [309] 26.438990 27.857714 17.653863 35.197832 25.321072 22.729542 24.035069
## [316] 21.334782 30.079757 17.220074 25.671010 11.796301 21.077821 24.549518
## [323]  3.773152 29.264954 22.376759 25.373723 20.206149 18.382702 19.736727
## [330] 22.346929 20.552158 30.662015 30.553659 29.081707 24.962191 25.579393
## [337] 24.000520 16.485162 25.413386 20.540975 22.978031 27.569537 26.341642
## [344] 24.545251 21.792125 24.532223 21.583664 24.011971 21.419361 25.713740
```

```
w1<-length(z1)
w1
```

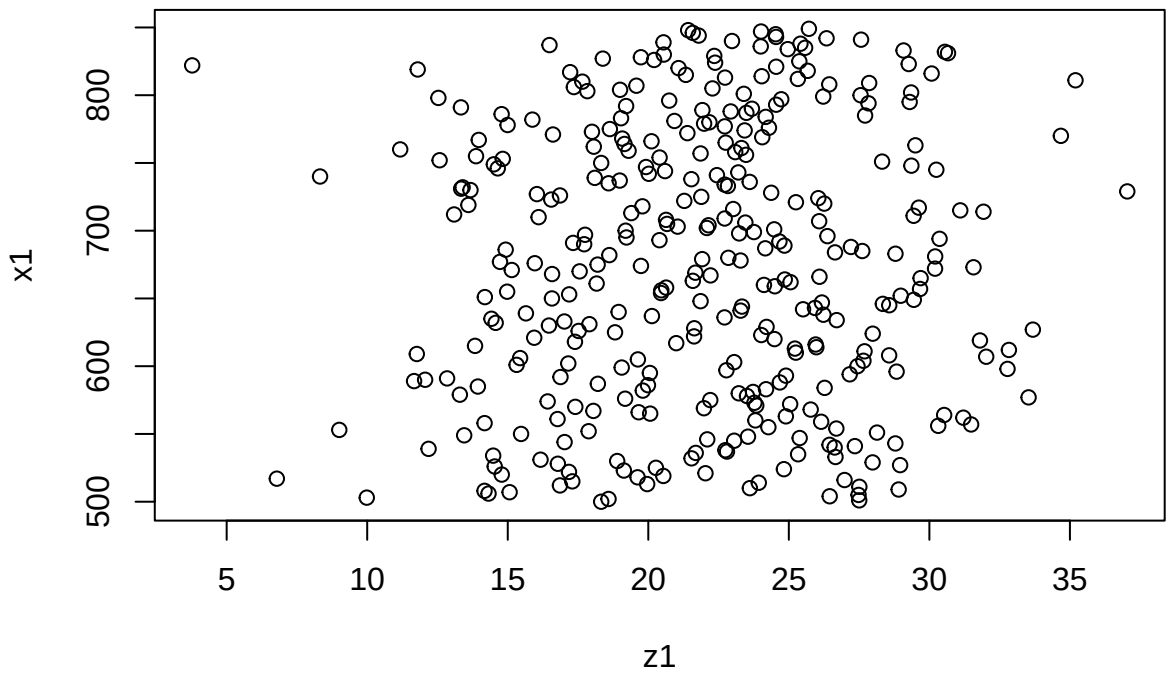
```
## [1] 350
```

```
x1<-(500:849)
x1
```

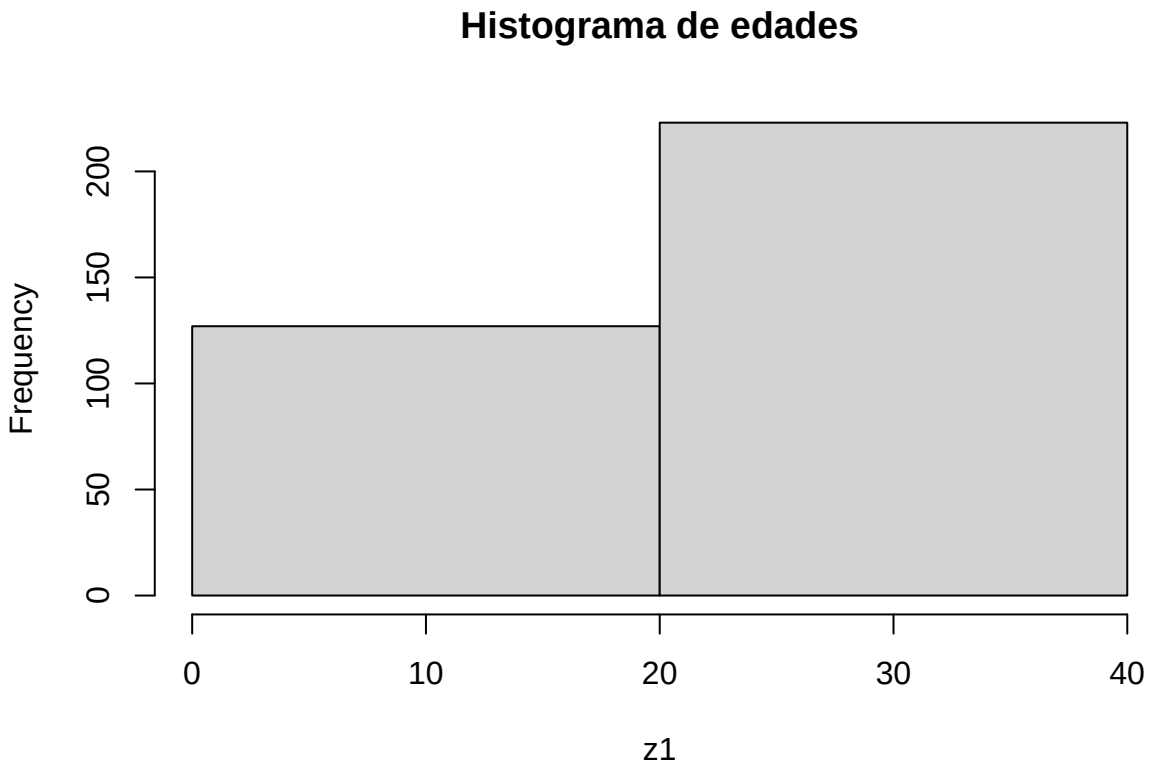
```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
```

```
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```




```
hist(z1,main="Histograma de edades",breaks=2)
```




```
density(z1,type='d')
```

```
## Warning: In density.default(z1, type = "d") :
```

```
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "d")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.509
##
##        x                 y
##  Min.   :-0.7538   Min.   :8.457e-06
##  1st Qu.: 9.8266   1st Qu.:1.166e-03
##  Median :20.4070   Median :1.124e-02
##  Mean   :20.4070   Mean   :2.358e-02
##  3rd Qu.:30.9874   3rd Qu.:4.604e-02
##  Max.   :41.5677   Max.   :7.162e-02
```

```
##plot(density(z1),type='d') ver si este plot va pq me ponia error
```

##Ejercicio 1 crear un vector que tenga todos los numeros enteros desde el 1 hasta 477

```
Secuencia_dni<-(1:477)
Secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477
```

##Ejercicio 2 calcular la suma de todos los valores del vector secuencia dni

```
Total<-0
Valor_final<-length(Secuencia_dni)
for (i in 1:Valor_final) {
  Total<-Total+i }
```

```
  Total
```

```
## [1] 114003
```

##Ejercicio 3 Repetir el ejercicio anterior en phyton

```
suma=0
for i in range (1,478):
  suma+=i
  print(suma)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
## 861
## 903
## 946
## 990
```

```
## 1035
## 1081
## 1128
## 1176
## 1225
## 1275
## 1326
## 1378
## 1431
## 1485
## 1540
## 1596
## 1653
## 1711
## 1770
## 1830
## 1891
## 1953
## 2016
## 2080
## 2145
## 2211
## 2278
## 2346
## 2415
## 2485
## 2556
## 2628
## 2701
## 2775
## 2850
## 2926
## 3003
## 3081
## 3160
## 3240
## 3321
## 3403
## 3486
## 3570
## 3655
## 3741
## 3828
## 3916
## 4005
## 4095
## 4186
## 4278
## 4371
## 4465
## 4560
## 4656
## 4753
## 4851
```

```
## 4950
## 5050
## 5151
## 5253
## 5356
## 5460
## 5565
## 5671
## 5778
## 5886
## 5995
## 6105
## 6216
## 6328
## 6441
## 6555
## 6670
## 6786
## 6903
## 7021
## 7140
## 7260
## 7381
## 7503
## 7626
## 7750
## 7875
## 8001
## 8128
## 8256
## 8385
## 8515
## 8646
## 8778
## 8911
## 9045
## 9180
## 9316
## 9453
## 9591
## 9730
## 9870
## 10011
## 10153
## 10296
## 10440
## 10585
## 10731
## 10878
## 11026
## 11175
## 11325
## 11476
## 11628
```

```
## 11781
## 11935
## 12090
## 12246
## 12403
## 12561
## 12720
## 12880
## 13041
## 13203
## 13366
## 13530
## 13695
## 13861
## 14028
## 14196
## 14365
## 14535
## 14706
## 14878
## 15051
## 15225
## 15400
## 15576
## 15753
## 15931
## 16110
## 16290
## 16471
## 16653
## 16836
## 17020
## 17205
## 17391
## 17578
## 17766
## 17955
## 18145
## 18336
## 18528
## 18721
## 18915
## 19110
## 19306
## 19503
## 19701
## 19900
## 20100
## 20301
## 20503
## 20706
## 20910
## 21115
## 21321
```

```
## 21528
## 21736
## 21945
## 22155
## 22366
## 22578
## 22791
## 23005
## 23220
## 23436
## 23653
## 23871
## 24090
## 24310
## 24531
## 24753
## 24976
## 25200
## 25425
## 25651
## 25878
## 26106
## 26335
## 26565
## 26796
## 27028
## 27261
## 27495
## 27730
## 27966
## 28203
## 28441
## 28680
## 28920
## 29161
## 29403
## 29646
## 29890
## 30135
## 30381
## 30628
## 30876
## 31125
## 31375
## 31626
## 31878
## 32131
## 32385
## 32640
## 32896
## 33153
## 33411
## 33670
## 33930
```

```
## 34191
## 34453
## 34716
## 34980
## 35245
## 35511
## 35778
## 36046
## 36315
## 36585
## 36856
## 37128
## 37401
## 37675
## 37950
## 38226
## 38503
## 38781
## 39060
## 39340
## 39621
## 39903
## 40186
## 40470
## 40755
## 41041
## 41328
## 41616
## 41905
## 42195
## 42486
## 42778
## 43071
## 43365
## 43660
## 43956
## 44253
## 44551
## 44850
## 45150
## 45451
## 45753
## 46056
## 46360
## 46665
## 46971
## 47278
## 47586
## 47895
## 48205
## 48516
## 48828
## 49141
## 49455
```

```
## 49770
## 50086
## 50403
## 50721
## 51040
## 51360
## 51681
## 52003
## 52326
## 52650
## 52975
## 53301
## 53628
## 53956
## 54285
## 54615
## 54946
## 55278
## 55611
## 55945
## 56280
## 56616
## 56953
## 57291
## 57630
## 57970
## 58311
## 58653
## 58996
## 59340
## 59685
## 60031
## 60378
## 60726
## 61075
## 61425
## 61776
## 62128
## 62481
## 62835
## 63190
## 63546
## 63903
## 64261
## 64620
## 64980
## 65341
## 65703
## 66066
## 66430
## 66795
## 67161
## 67528
## 67896
```

```
## 68265
## 68635
## 69006
## 69378
## 69751
## 70125
## 70500
## 70876
## 71253
## 71631
## 72010
## 72390
## 72771
## 73153
## 73536
## 73920
## 74305
## 74691
## 75078
## 75466
## 75855
## 76245
## 76636
## 77028
## 77421
## 77815
## 78210
## 78606
## 79003
## 79401
## 79800
## 80200
## 80601
## 81003
## 81406
## 81810
## 82215
## 82621
## 83028
## 83436
## 83845
## 84255
## 84666
## 85078
## 85491
## 85905
## 86320
## 86736
## 87153
## 87571
## 87990
## 88410
## 88831
## 89253
```

```
## 89676
## 90100
## 90525
## 90951
## 91378
## 91806
## 92235
## 92665
## 93096
## 93528
## 93961
## 94395
## 94830
## 95266
## 95703
## 96141
## 96580
## 97020
## 97461
## 97903
## 98346
## 98790
## 99235
## 99681
## 100128
## 100576
## 101025
## 101475
## 101926
## 102378
## 102831
## 103285
## 103740
## 104196
## 104653
## 105111
## 105570
## 106030
## 106491
## 106953
## 107416
## 107880
## 108345
## 108811
## 109278
## 109746
## 110215
## 110685
## 111156
## 111628
## 112101
## 112575
## 113050
## 113526
```

```
## 114003
```

##Ejercicio 4 Cuanto tarda en correr el codigo del ejercicio 2?

```
inicio<-Sys.time()
Total<-0
Valor_final<-length(Secuencia_dni)
for (i in 1:Valor_final) {
  Total<-Total+i }
  Total
```

```
## [1] 114003
```

```
  Sys.time()
```

```
## [1] "2026-05-18 22:53:34 UTC"
```

```
  final<-Sys.time()
  final-inicio
```

```
## Time difference of 0.004731894 secs
```

##Ejercicio 5 Repetir el ejercicio anterior usando la cabeza como Gauss

```
n=833
suma=(n*(n+1))/2
print(suma)
```

```
## [1] 347361
```

####PARTE 2 ##1.4 CONSIGNA: Comparar la performance con systime

```
A <- 0
InicioA=Sys.time()
for (i in 1:50000) { A[i] <- (i*2)}
head (A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```
FinA=Sys.time()
FinA-InicioA
```

```
## Time difference of 0.02285624 secs
```

```
InicioB=Sys.time()
B=seq(1, 100000, by = 2)
head(B)
```

```
## [1]  1  3  5  7  9 11
```

```
tail(B)
```

```
## [1] 99989 99991 99993 99995 99997 99999
```

```
FinB=Sys.time()
FinB-InicioB
```

```
## Time difference of 0.002487183 secs
```

##1.5 CONSIGNA: Implementacion de una serie Fibonachi o Fibonacci

```r
n <- 30
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
  for(i in 3:n){
    fib[i] <- fib[i-1]+fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.003   0.000   0.003
```

```r
print("primeros numeros de la serie ")
```

```
## [1] "primeros numeros de la serie "
```

```r
head(fib,10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```r
fib_recursiva<- function(k){
  if(k==0)return(0)
  if(k==1)return(1)
  return(fib_recursiva(k-1)+fib_recursiva(k-2))
}
print("tiempo de ejecucion recursiva:")
```

```
## [1] "tiempo de ejecucion recursiva:"
```

```r
system.time({
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   0.794   0.002   0.807
```

##1.6 CONSIGNA: Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000

```r
a0=0
a1=1
it=2
  while (a1<1000001){
    an=a0+a1
    a0=a1
    a1=an
    it=it+1}
cat("el primer numero mayor a 1000000 es:",a1,"/n")
```

```
## el primer numero mayor a 1000000 es: 1346269 /n
```

```r
cat("se necesitaron", it-1,"iteraciones (pasos) para superarlo.")
```

```
## se necesitaron 31 iteraciones (pasos) para superarlo.
```

```r
it
```

```
## [1] 32
```

##1.7 CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
library(microbenchmark)
x <- sample(1:20000,10)
# Creo una funcion para ordenar
burbuja <- function(x){
  res <- microbenchmark(NULL, burbuja, times=1000L)
n<-length(x)
for(j in 1:(n-1)){
for(i in 1:(n-j)){
if(x[i]>x[i+1]){
temp<-x[i]
x[i]<-x[i+1]
x[i+1]<-temp
}
}
}
return(x)
}
res<-burbuja(x)
#Muestra obtenida
x
```

```
##  [1] 13785  1903  5219 17334 11055  9436  9804 18148 18780  9066
```

```
res
```

```
##  [1]  1903  5219  9066  9436  9804 11055 13785 17334 18148 18780
```

```
sort(x)
```

```
##  [1]  1903  5219  9066  9436  9804 11055 13785 17334 18148 18780
```

##1.8 CONSIGNA: Penitencia de Newton

```
n <- 1e7
system.time({
  suma_iterativa <- 0
  for(i in 1:n){
  suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##   0.163   0.000   0.163
```

```
cat("Resultado suma:",suma_iterativa, "/n")
```

```
## Resultado suma: 5e+13 /n
```

```
system.time({
  suma_analitica<- (n*(n+1))/2
})
```

```
##    user  system elapsed
##       0       0       0
```

```
cat("Resultado suma analitico",suma_analitica,"/n")
```

```
## Resultado suma analitico 5e+13 /n
```

Investigar con los gráficos de violin (de la biblioteca micro como se comporta una computadora usando métodos de ordenamiento "burbuja" y compararlo con el método sort nativo de R .

Medir la performance del método kmean de agrupamiento. Realizar una inroducción teórica antes de medir con gráfico de violin.

```
library(microbenchmark)
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:2000, 200)

# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 20
)

# Convertir a ms
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
  geom_violin(trim = FALSE, fill = "lightblue", alpha = 0.6) +
```

```
  geom_boxplot(width = 0.1, outlier.shape = NA) +
  labs(title = "Bubble Sort vs sort()",
       x = "Método",
       y = "Tiempo (ms)") +
  theme_minimal()
```